```
        for (int i = startIdx + 1; i < t.length; i++) {
            if (t[i] < min) {
                min = t[i];
                rVal = i;
            }
        }
        return rVal;
    }

    public static void sort(int[] t) {
        for (int i = 0; i < t.length; i++) {
            int mIdx = idxOfMin(t, i);
            if (mIdx != i) {
                swap(t, mIdx, i);
            }
        }
    }

}
```

Κώδικας 7.25 *Η κλάση MyArrays*

Οι δύο πρώτες συναρτήσεις είναι απλές. Η πρώτη κάνει σειριακή αναζήτηση σε πίνακα ακέραιων και η δεύτερη σε πίνακα αλφαριθμητικών σειρών. Η μόνη λεπτομέρεια που πρέπει να προσέξουμε είναι πως για μεν τους ακέραιους, η σύγκριση γίνεται με τον τελεστή ==, για δε τις αλφαριθμητικές σειρές, η σύγκριση γίνεται με τη συνάρτηση equals.

Η cntOccurances επίσης βασίζεται σε σειριακή προσπέλαση όλων των στοιχείων του πίνακα. Κάθε φορά που βρίσκει τιμή ίση με τη δεύτερη παράμετρό της, αυξάνει έναν μετρητή τον οποίον και επιστρέφει.

Η toString για μονοδιάστατο πίνακα ακέραιων, καταρχάς επιστρέφει [] αν η παράμετρός της έχει μήκος μηδέν. Στη συνέχεια, αρχικοποιεί την τοπική μεταβλητή s στη σειρά [. Μετά προσπελαύνει ένα προς ένα τα στοιχεία τα στοιχεία του t και προσθέτει στο s την τιμή του στοιχείου ακολουθούμενη από κόμμα και διάστημα. Εξαιρεί όμως την πρόσθεση κόμματος και διαστήματος στο τελευταίο στοιχείο. Τέλος, κλείνει το s προσθέτοντας το].

Η toString για δισδιάστατο πίνακα έχει ανάλογη λογική με την toString για μονοδιάστατο. Μόνο που εδώ τα στοιχεία είναι πίνακες, οπότε χρησιμοποιεί την toString για να πάρει την αναπαράσταση των μονοδιάστατων πινάκων στοιχείων του t.

Η reverse ορίζει τον πίνακα rVal με μήκος ίσο με το μήκος του πίνακα input. Στη συνέχεια αντιγράφει ένα προς ένα τα στοιχεία του input στον rVal σε αντιδιαμετρικά αντίθετη θέση. Προσέξτε πως όταν ο δείκτης i είναι ίσος με μηδέν, δηλαδή αναφερόμαστε στην πρώτη θέση του input, η έκφραση rVal.length-1-i αναφέρεται στην τελευταία θέση του rVal, όταν ο δείκτης i είναι ίσος με 1, δηλαδή αναφερόμαστε στη δεύτερη θέση του input, η έκφραση rVal.length-1-i αναφέρεται στην προτελευταία θέση του rVal, κοκ.

Η sum εφαρμόζει σειριακή προσπέλαση και προσθέτει κάθε τιμή του t στην τοπική μεταβλητή sum την οποία τελικά επιστρέφει.

Παρόμοια η max εφαρμόζει σειριακή προσπέλαση. Αρχικοποιεί την τοπική μεταβλητή max στην τιμή στη θέση μηδέν του t. Στη συνέχεια, επισκέπτεται τα στοιχεία του t από τη θέση 1 και μετά και κάθε φορά που βρίσκει μεγαλύτερη τιμή ενημερώνει την max και την idx που κρατάει τη θέση της max. Επιστρέφει την idx.

Η equals για πίνακες ακέραιων συγκρίνει καταρχάς τα μήκη των πινάκων. Αν δεν είναι ίσα, επιστρέφει false. Αν δεν επιστρέψει από τον έλεγχο των μηκών, ψάχνει στη συνέχεια αν κάθε στοιχείο του tA είναι και στοιχείο του tB. Προσέξτε πως ο αντίστροφος έλεγχος είναι περιττός από τη στιγμή που τα μήκη είναι ίσα.

Ίδια είναι και η λογική της equals για πίνακες από Strings.

Η binarySearch εφαρμόζει τη λογική που αναπτύξαμε στην ενότητα 6.5.2.

Η swap κρατάει την πρώτη από τις τιμές που αντιμεταθέτει σε προσωρινή μεταβλητή, εκχωρεί τη δεύτερη τιμή στην πρώτη και στη θέση της δεύτερης τιμής εκχωρεί την τιμή της προσωρινής μεταβλητής.

Η sort υλοποιεί τον αλγόριθμο που παρατίθεται στον κώδικα 6.18. Η διαφορά της είναι πως την εύρεση του μικρότερου στοιχείου την αναθέτει στην idxMin.

7.9.6 Ταξινόμηση δισδιάστατου πίνακα με βάση το άθροισμα γραμμών

Να υλοποιηθεί συνάρτηση με διεπαφή public static void sortSubArraysBySum(int[][] t) που ταξινομεί τους πίνακες του t κατά αύξουσα αριθμητική σειρά του αθροίσματος των στοιχείων τους. Για παράδειγμα, ο πίνακας

```
{
  {100, 200, 300, 400},
  {1, 2, 3, 4, 5},
  {10, 20, 30, 40, 50, 60}
}
```

θα πρέπει να ταξινομηθεί ως

```
{
   {1, 2, 3, 4, 5},
   {10, 20, 30, 40, 50, 60},
   {100, 200, 300, 400}
}
```

καθώς 1+2=3+4+5 < 10+20+30+40+50+60 < 100+200+300+400΄

Λύση
Θα εφαρμόσουμε παρόμοια λογική με αυτήν της ταξινόμησης μονοδιάστατου πίνακα. Έτσι, θα χρειαστούμε μια συνάρτηση που εντοπίζει τον πίνακα-στοιχείο του t που έχει το μικρότερο άθροισμα.

```
static int idxOfMinSum(int[][] t, int startIdx) {
    int min = MyArrays.sum(t[startIdx]);
    int rVal = startIdx;
    for (int i = startIdx + 1; i < t.length; i++) {
        if (MyArrays.sum(t[i]) < min) {
            min = MyArrays.sum(t[i]);
            rVal = i;
        }
    }
    return rVal;
}
```

Κώδικας 7.26 *Εύρεση πίνακα στοιχείου με το μικρότερο άθροισμα*

Ο κώδικας 7.26 παρουσιάζει την idxOfMinSum που αρχίζοντας μετά από τη θέση startIdx εντοπίζει ποιος πίνακας-στοιχείο του t έχει το μικρότερο άθροισμα στοιχείων.
Έχοντας στη διάθεσή μας την idxOfMinSum, μπορούμε να υλοποιήσουμε την απαιτούμενη ταξινόμηση εφαρμόζοντας αντίστοιχη λογική με αυτήν της ταξινόμησης μονοδιάστατου πίνακα. Ο κώδικας 7.27 παρουσιάζει τη σχετική υλοποίηση:

```
public static void sortSubArraysBySum(int[][] t) { //Κώδικας 7.27
    for (int i = 0; i < t.length; i++) {
        int mIdx = idxOfMinSum(t, i);
        if (mIdx != i) {
            int[] local = t[i];
            t[i] = t[mIdx];
            t[mIdx] = local;
        }
    }
}
```

Κώδικας 7.27 *Ταξινόμηση σύμφωνα με το άθροισμα των γραμμών δισδιάστατου πίνακα*

7.9.7 Διαχείριση ψηφιοσειρών

Να υλοποιηθεί η κλάση BitMaps που περιλαμβάνει τις λειτουργίες

```
public static int setBit(int i, int p)
```
Θέτει στο bit που βρίσκεται στη θέση p αντιγράφου του ακέραιου i την τιμή 1. Επιστρέφει τον παραγόμενο ακέραιο.

```
public static int resetBit(int i, int p)
```
Θέτει στο bit που βρίσκεται στη θέση p αντιγράφου του ακέραιου i την τιμή 0. Επιστρέφει τον παραγόμενο ακέραιο.

```
public static int getBit(int i, int p)
```
Επιστρέφει την τιμή του bit που βρίσκεται στη θέση p του ακέραιου i

```
public static int reverseBit(int i, int p)
```
Αντιστρέφει σε αντίγραφο του i το bit στη θέση p. Επιστρέφει τον παραγόμενο ακέραιο.

Λύση
Παρουσιάζουμε τον κώδικα της κλάσης BitMap ακολουθούμενο από τις αναγκαίες επεξηγήσεις.

```
public class BitMap { //Κώδικας 7.28

    public static int setBit(int i, int p) {
        int local = 1;
        local <<= p;
        return i | local;
    }

    public static int resetBit(int i, int p) {
        int local = setBit(0, p);
        local = ~local;
        return i & local;
    }

    public static int getBit(int i, int p) {
        int local = setBit(0, p);
        if ((local & i) == local) {
            return 1;
        }
        return 0;
    }

    public static int reverseBit(int i, int p) {
        if (getBit(i, p) == 1) {
            return resetBit(i, p);
        } else {
            return setBit(i, p);
        }
    }

    public static void main(String[] args) {
        System.out.println(setBit(1, 1));
        System.out.println(resetBit(3, 0));
        System.out.println(getBit(3, 1));
        System.out.println(getBit(3, 2));
        System.out.println(reverseBit(3,0));
    }
}
```

Κώδικας 7.28 *Διαχείριση ψηφιοσειρών*

Η συνάρτηση setBit, στην πρώτη γραμμή αρχικοποιεί τη μεταβλητή local στην τιμή 1. Επομένως, ο ψηφιοχάρτης της local γίνεται 00000000000000000000000000000001, δηλαδή 31 μηδενικά ακολουθούμενα από έναν άσο. Παρότι ο int έχει μήκος 32 bits, για απλοποίηση, στο εξής θα θεωρούμε ότι είναι μήκους 8 bits. Με αυτήν την υπόθεση, η local έχει το ακόλουθο περιεχόμενο 00000001. Στη συνέχεια εκτελείται διολίσθηση στα αριστερά για όσες θέσεις καθορίζει η παράμετρος p. Για παράδειγμα, θεωρήστε την κλήση setBit(1,1) στην πρώτη γραμμή της main. Η p εδώ έχει την τιμή 1. Επομένως, η local θα αποκτήσει περιεχόμενο 00000010. Στη συνέχεια γίνεται bitwise OR μεταξύ της παραμέτρου i και της local. Η παράμετρος i έχει τιμή 1, δηλαδή περιεχόμενο 00000001. Συνεπώς 00000010 | 00000001=00000011, δηλαδή 3 στο δεκαδικό σύστημα που είναι και η τιμή που εμφανίζεται από την println στην πρώτη γραμμή της main.

Με άλλα λόγια διαμορφώσαμε έναν ψηφιοχάρτη στη μεταβλητή local ο οποίος έχει όλα τα bits ίσα με το 0, εκτός από το bit που θέλουμε να κάνουμε 1. Στη συνέχεια, το bitwise OR μεταξύ της local και οποιουδήποτε ακέραιου, x, θα μετατρέψει σε 1 το bit του x που βρίσκεται στη θέση που έχει η local τιμή 1.

Παρόμοια εργαζόμαστε και για την resetBit. Ορίζουμε τη μεταβλητή local που έχει όλα τα bits 0 εκτός από το bit στη θέση p. Με bitwise not αντιστρέφουμε όλα τα bits της local οπότε σε όλες τις θέσεις έχει πλέον 1 εκτός από τη θέση p που έχει 0. Στη συνέχεια το bitwise and με την παράμετρο i, έχει ως αποτέλεσμα όλα τα bits να παραμείνουν ως έχουν εκτός από το bit στη θέση p που αποκτά την τιμή 0.

Η getBit δημιουργεί τον ακέραιο local που αρχικοποιείται ώστε ο ψηφιοχάρτης του να αποτελείται από 0 εκτός από τη θέση p. Επομένως, αν το bitwise and μεταξύ της local και της i επιστρέψει ακέραιο ίσο με την local αυτό σημαίνει πως το bit στη θέση p της i είναι 1 αλλιώς 0.

Τέλος η reverseBit είναι απλή καθώς βασίζεται στις getBit, resetBit και setBit και δεν κάνει απευθείας πράξεις σε επίπεδο ψηφίου.

7.10 Ασκήσεις προς λύση

1. Να αναπτύξετε συνάρτηση double abs(double n) που επιστρέφει την απόλυτη τιμή του n.

2. Να αναπτύξετε συνάρτηση που υπολογίζει και τυπώνει την τιμή της $f(x)=x^4+5x^3+2x^2+8x+1$ για τις τιμές του x=1..100, όπου x ακέραιος.

3. Να αναπτύξετε συνάρτηση που τυπώνει τους ν πρώτους όρους της ακολουθίας Fibonacci.

4. Να αναπτυχθεί συνάρτηση που επιστρέφει true αν η ακέραιη παράμετρός της είναι πρώτος αριθμός.

5. Να αναπτυχθεί συνάρτηση που τυπώνει τους πρώτους αριθμούς από το 1 έως το 100.

6. Να αναπτυχθεί συνάρτηση που τυπώνει τους αριθμούς από το 1 έως το 100 που δεν είναι πρώτοι.

7. Να αναπτυχθεί συνάρτηση που λαμβάνει μια boolean παράμετρο και επιστρέφει έναν πίνακα με τους χαρακτήρες της Αγγλικής, κεφαλαίους ή πεζούς, ανάλογα με την τιμή της παραμέτρου.

8. Να υλοποιηθεί Java application που λαμβάνει 3 παραμέτρους που αντιπροσωπεύουν μια εξίσωση 2ου βαθμού ($ax^2+bx+c=0$) και τυπώνει τη λύση της εφόσον υπάρχει ή διαφορετικά το μήνυμα δεν υπάρχει λύση, διακρίνουσα <0.

9. Να αναπτυχθεί συνάρτηση που επιστρέφει το άθροισμα των ψηφίων ενός ακεραίου.

10. Να αναπτυχθεί συνάρτηση με διεπαφή static int max(int…in) που λαμβάνει μια σειρά από ακέραιους και επιστρέφει τον μεγαλύτερο.

11. Να αναπτυχθεί συνάρτηση που λαμβάνει ως παράμετρο έναν πραγματικό αριθμό που αναπαριστά θερμοκρασία σε Celsius και επιστρέφει τη θερμοκρασία σε Fahrenheit.

12. Υπερφορτώστε την equals της άσκησης 7.8.5 ώστε να εξετάζει και πίνακες πραγματικών.

13. Αναπτύξτε συνάρτηση με διεπαφή static int[] resize(int[] t, int newSize) που επιστρέφει ένα αντίγραφο του t με μήκος newSize. Αν η τιμή της newSize είναι μικρότερη από το αρχικό μήκος του t, τότε ενδέχεται να απωλεσθούν δεδομένα από τον πίνακα.

14. Οποιοσδήποτε βαθμός μικρότερος του 5 αντιστοιχεί σε αποτυχία. Η βαθμολογία από 5 έως 6,4 αντιστοιχεί στον χαρακτηρισμό «καλώς», από 6,5 έως 8,4 στο «λίαν καλώς» και από 8,5 έως 10 στο «άριστα». Να

αναπτύξετε συνάρτηση που λαμβάνει ως είσοδο έναν βαθμό από 0 έως 10 και επιστρέφει τον αντίστοιχο χαρακτηρισμό.

15. Να αναπτυχθεί εφαρμογή Java που παράγει 100 τυχαίους αριθμούς από το 1 έως το 40 και στη συνέχεια μετράει.

- Πόσοι από τους παραγόμενους αριθμούς είναι μοναδικοί.
- Πόσοι είναι άρτιοι και πόσοι περιττοί.
- Ποιοι αριθμοί από το 1 έως το 40 δεν υπάρχουν καθόλου στην παραγόμενη ακολουθία.
- Ποιος ή ποιοι αριθμοί εμφανίζονται περισσότερες φορές.

20. Να αναπτυχθεί συνάρτηση με διεπαφή static void reverse(char[] t) που αντιστρέφει τις θέσεις των στοιχείων του t, δηλαδή τοποθετεί το στοιχείο από την πρώτη στην τελευταία θέση του πίνακα, από τη δεύτερη στην προτελευταία, κοκ.

21. Να αναπτυχθεί συνάρτηση που επιστρέφει τον μέγιστο κοινό διαιρέτη ενός πλήθους ακέραιων.

22. Να αναπτυχθεί συνάρτηση που επιστρέφει το ελάχιστο κοινό πολλαπλάσιο ενός πλήθους ακέραιων.

23. Να αναπτύξετε συνάρτηση που λαμβάνει ως παράμετρο έναν πίνακα ακέραιων και επιστρέφει επίσης έναν πίνακα ακέραιων που αναπαριστά το σύνολο των ακέραιων του πίνακα εισόδου. Προσέξτε πως σε ένα σύνολο, ένα στοιχείο μπορεί να υπάρχει μόνο μία φορά.

24. Να αναπτύξετε συνάρτηση που υλοποιεί την ταξινόμηση φυσαλίδας σε πίνακα πραγματικών.

25. Να αναπτύξετε συνάρτηση που εξετάζει αν ένας πίνακας είναι ταξινομημένος. Η συνάρτηση να επιστρέφει 0 για αταξινόμητους πίνακες, -1 για ταξινομημένους κατά φθίνουσα σειρά και 1 για ταξινομημένους κατά αύξουσα σειρά.

26. Να αναπτύξετε συνάρτηση που συγχωνεύει δύο ταξινομημένους πίνακες σε έναν νέο πίνακα επίσης ταξινομημένο.

27. Να αναπτύξετε συνάρτηση που λαμβάνει ως παράμετρο δύο μονοδιάστατους πίνακες ακέραιων και επιστρέφει έναν πίνακα που περιέχει μόνο τα κοινά στοιχεία των παραμέτρων της έτσι ώστε κάθε κοινό στοιχείο να υπάρχει μόνο μία φορά στον πίνακα επιστροφής.

28. Να αναπτύξετε συνάρτηση που λαμβάνει ως παράμετρο δύο τετραγωνικούς πίνακες πραγματικών και επιστρέφει το άθροισμά τους. Σημειώστε πως τετραγωνικοί λέγονται οι πίνακες που έχουν ίδιο αριθμό γραμμών και στηλών.

29. Να αναπτύξετε συνάρτηση που ανακατανέμει τα στοιχεία ενός μονοδιάστατου πίνακα.

30. Να αναπτύξετε συνάρτηση που ανακατανέμει τα στοιχεία ενός δισδιάστατου πίνακα έτσι ώστε ένα τελικό στοιχείο να είναι πιθανό να βρεθεί σε οποιαδήποτε γραμμή και στήλη του πίνακα.

31. Ο τόκος ενός κεφαλαίου υπολογίζεται με βάση το επιτόκιο. Όταν ο τόκος μίας περιόδου προστίθεται στο κεφάλαιο, τότε ο τόκος της επόμενης περιόδου υπολογίζεται επί του αρχικού κεφαλαίου αυξημένου κατά τον τόκο της προηγούμενης περιόδου. Το φαινόμενο αυτό ονομάζεται ανατοκισμός. Να αναπτύξετε συνάρτηση που λαμβάνει ως παραμέτρους το κεφάλαιο, το επιτόκιο και τον αριθμό των περιόδων και υπολογίζει το σύνολο του τόκου.

32. Να αναπτυχθεί συνάρτηση που επιστρέφει την κύρια διαγώνιο τετραγωνικού πίνακα.

33. Να αναπτυχθεί συνάρτηση που επιστρέφει το Ελάχιστο Κοινό Πολλαπλάσιο μιας σειράς ακέραιων.

34. Να αναπτυχθεί συνάρτηση που επιστρέφει τον Μέγιστο Κοινό Διαιρέτη μιας σειράς ακέραιων.

35. Έστω nb ένας αριθμός εκφρασμένος σε θεσιακό αριθμητικό σύστημα με βάση b. Η μετατροπή του στο δεκαδικό σύστημα αρίθμησης γίνεται από τον τύπο

$$n_b = s_k \times b^k + s_{k-1} \times b^{k-1} + \cdots + s_1 \times b^1 + s_0 \times b^0 + s_{-1} \times b^{-1} + s_{-2} \times b^{-2} + \cdots + s_m \times b^m$$

Θεωρήστε ως θεσιακά αριθμητικά συστήματα μόνο αυτά που μας ενδιαφέρουν στην Πληροφορική, δηλαδή το δυαδικό, το οκταδικό και το δεκαεξαδικό.

36. Να υλοποιηθεί συνάρτηση που λαμβάνει δύο παραμέτρους τύπου String. Η πρώτη αναπαριστά έναν αριθμό που μπορεί να περιέχει ή όχι, υποδιαιρέσεις της μονάδας, π.χ. 101101.98, 25634.01, AB32.A ή 101101, 25634, AB32. Η δεύτερη αναπαριστά τη βάση του αριθμητικού συστήματος στο οποίο εκφράζεται ο αριθμός και μπορεί να λάβει τις τιμές 2, 8 ή 16. Η συνάρτηση επιστρέφει την τιμή του αριθμού στο δεκαδικό σύστημα ως double.

Καλέστε τη συνάρτηση και μετατρέψτε τους ακόλουθους αριθμούς στο δεκαδικό σύστημα.

```
110111102
65128
A2E3416
10110.0112
763.2348
A2E34.A16
BB135.1B16
```

Βρείτε κατάλληλο υπολογιστή μετατροπών στο διαδίκτυο και ελέγξτε τα αποτελέσματα.

Κεφάλαιο 8

Σύνοψη
Σε αυτήν την ενότητα παρουσιάζονται τα βασικά στάδια της ανάπτυξης εφαρμογών προσαρμοσμένα στις γνώσεις που έχει αποκομίσει ο αναγνώστης μέχρι αυτό το σημείο του βιβλίου. Ο στόχος είναι να διευκολυνθεί ο αρχάριος προγραμματιστής να υιοθετήσει έναν δομημένο τρόπο ανάλυσης, σχεδίασης και υλοποίησης τόσο των ασκήσεων του βιβλίου όσο και γενικότερων εφαρμογών. Προς αυτόν τον στόχο παρουσιάζονται οι έννοιες του δομημένου προγραμματισμού, τα στάδια της ανάπτυξης, η συλλογή απαιτήσεων, η ανάλυση και ο σχεδιασμός, η τεκμηρίωση και απολαθοποίηση του κώδικα καθώς και οι διαδικασίες και τα εργαλεία ελέγχου της ορθότητάς του.

Προαπαιτούμενη γνώση
Οι θεμελιώδεις τύποι και βασική γνώση του τύπου String. Βασική είσοδος και έξοδος. Η λειτουργία των τελεστών. Οι δομές επιλογής και οι επαναληπτικές δομές. Η διαχείριση των πινάκων. Οι συναρτήσεις, η έννοια και βασική διαχείριση των απροσδόκητων λαθών, οι προσδιοριστές προσπέλασης και η χρήση των πακέτων.

Λέξεις κλειδιά
Δομημένος προγραμματισμός, απαιτήσεις, ανάλυση, σχεδιασμός, τεκμηρίωση, απολαθοποίηση.

8 Ανάπτυξη εφαρμογών

Η ανάπτυξη των εφαρμογών δεν αρχίζει ούτε τελειώνει με τη συγγραφή του κώδικα σε κάποια γλώσσα προγραμματισμού. Αντίθετα, μια σειρά από διαδικασίες που λαμβάνουν χώρα πριν, μετά και κατά τη διάρκεια της συγγραφής του κώδικα διαδραματίζουν κρίσιμο ρόλο στην ανάπτυξη μιας ολοκληρωμένης εφαρμογής που απαντά αποτελεσματικά στις ανάγκες των χρηστών της. Στις διαδικασίες αυτές συμπεριλαμβάνονται κατ' ελάχιστο, η συλλογή και ανάλυση των απαιτήσεων, ο σχεδιασμός της εφαρμογής, η κωδικοποίηση και απολαθοποίηση, η τεκμηρίωση του κώδικα και ο έλεγχος της ορθότητάς του.

Αυτές τις διαδικασίες παρουσιάζει αυτή η ενότητα προσαρμοσμένες στις ανάγκες μιας εισαγωγής στον προγραμματισμό. Ο σκοπός της ενότητας δεν είναι να αντικαταστήσει τη μελέτη των διάφορων τεχνικών ανάλυσης και σχεδίασης λογισμικού. Εξάλλου η μελέτη αυτών των τεχνικών είναι έξω από την εμβέλεια αυτού του βιβλίου. Ωστόσο, κάποιες κατευθυντήριες γραμμές είναι χρήσιμες και μπορεί να βοηθήσουν στην αποτελεσματική ανάπτυξη ορθότερων και δομημένων εφαρμογών ήδη από το στάδιο της εισαγωγής στον Προγραμματισμό.

Με βάση αυτό το σκεπτικό, σε αυτήν την ενότητα παρουσιάζεται μια απλοποιημένη εκδοχή των διαδικασιών συλλογής και ανάλυσης απαιτήσεων και σχεδιασμού λογισμικού. Οι υπόλοιπες διαδικασίες, δηλαδή η υλοποίηση, η απολαθοποίηση, η τεκμηρίωση και ο έλεγχος ορθότητας του προγράμματος, παρουσιάζονται στο πλαίσιο της Java, αξιοποιούν εργαλεία που σχετίζονται άμεσα με την Java και εκτίθενται με μεγαλύτερη λεπτομέρεια. Δεν θα πρέπει όμως να θεωρηθεί ότι τα θέματα που πραγματεύεται το κεφάλαιο αυτό αναπτύσσονται σε πλήρη έκταση. Ο αναγνώστης που επιθυμεί να εμβαθύνει θα πρέπει να ανατρέξει σε σχετική βιβλιογραφία. Ας εξηγήσουμε όμως καταρχάς την έννοια του δομημένου προγράμματος

8.1 Δομημένος Προγραμματισμός

Οι στόχοι του Δομημένου Προγραμματισμού (Structured Programming) είναι να καταστήσει τα προγράμματα ευανάγνωστα, να περιορίσει τα λάθη σε αυτά και να επιταχύνει τον χρόνο ανάπτυξης. Το κύριο συστατικό του είναι η ρουτίνα, δηλαδή το γνωστό μας υποπρόγραμμα ή η συνάρτηση στο πλαίσιο της Java. Η ρουτίνα θα πρέπει να είναι αυτόνομη και να δίνει απάντηση σε ένα σαφώς ορισμένο πρόβλημα, ενώ ο συνδυασμός των ρουτίνων θα πρέπει να γίνεται με τρόπο που δομημένα ανακλά το πρόβλημα που αποτελεί το κίνητρο της ανάπτυξης.

Ο Δομημένος Προγραμματισμός προέκυψε τη δεκαετία του 1950 με την ανάπτυξη των γλωσσών Algol [1] και σύντομα επικράτησε τόσο στην ακαδημαϊκή κοινότητα όσο και στον χώρο των επαγγελματιών προγραμματιστών. Η τεχνολογία του προγραμματισμού εκείνη την εποχή περιλάμβανε περίπου τις έννοιες που έχουν παρουσιαστεί μέχρι εδώ σε αυτό το βιβλίο. Ωστόσο, ο δομημένος προγραμματισμός κάθε άλλο παρά ξεπερασμένο μοντέλο θεωρείται. Αντίθετα αποτελεί θεμελιώδες μοντέλο προγραμματισμού, καθώς ενυπάρχει

στα περισσότερα πιο σύγχρονα μοντέλα όπως ο Αντικειμενοστρεφής Προγραμματισμός ή ο Προγραμματισμός Πρακτόρων (Agent Oriented Programming).

Αρχικά το μοντέλο βασίστηκε στο θεώρημα του δομημένου προγράμματος (structured program theorem) [2] σύμφωνα με το οποίο κάθε πρόγραμμα μπορεί να αναπτυχθεί συνδυάζοντας ρουτίνες με τρεις μόνο βασικές δομές.

1. Σειριακή εκτέλεση, δηλαδή η μία ρουτίνα εκτελείται σειριακά μετά από μία άλλη
2. Επιλογή, δηλαδή ο έλεγχος μιας λογικής συνθήκης αποφασίζει ποια ρουτίνα θα εκτελεστεί
3. Επανάληψη, δηλαδή μια ή περισσότερες ρουτίνες που συνδυάζονται σειριακά ή με επιλογή μπορούν να επαναλαμβάνονται κάτω από τον έλεγχο μιας λογικής συνθήκης.

Κάποιοι συγγραφείς προσθέτουν και την αναδρομή, δηλαδή τη δομή κατά την οποία μια ρουτίνα καλεί τον εαυτό της κάτω από μία συνθήκη ελέγχου. Ωστόσο, η αναδρομή δύσκολα μπορεί να θεωρηθεί βασική δομή, καθώς τα προβλήματα στα οποία αξιοποιείται μπορούν να επιλυθούν και με χρήση των τριών προαναφερόμενων δομών.

Σύντομα, η αποδοχή του μοντέλου έλαβε καθολικές διαστάσεις. Χαρακτηριστικό δείγμα η εξαφάνιση εντολών goto από τα προγράμματα παγκοσμίως. Οι εντολές τύπου goto αποτελούν ανεξέλεγκτες μεταπηδήσεις (unconditional jumps) από ένα σημείο του προγράμματος σε ένα άλλο, οπότε παραβιάζουν το θεώρημα του δομημένου προγράμματος, ενώ στην πράξη αποδεικνυόταν πως συχνά αποτελούσαν πηγή λαθών.

Στην πορεία στο μοντέλο έγιναν κάποιες παραλλαγές με κυριότερες την προσθήκη διαχείρισης εξαιρέσεων (Exceptions) και την πρόωρη έξοδο (early exit) από επαναληπτικές δομές και ρουτίνες.

Ο μηχανισμός των εξαιρέσεων αναπτύχθηκε τη δεκαετία του 1960 [3] ως μηχανισμός διαχείρισης απροσδόκητων γεγονότων που είναι πιθανό να συμβούν κατά την εκτέλεση του προγράμματος, π.χ. ένα πρόγραμμα επιχειρεί να ανοίξει ένα αρχείο για να επεξεργαστεί τα δεδομένα του αλλά το αρχείο δεν βρίσκεται στην καθορισμένη θέση στον δίσκο. Οι εξαιρέσεις συνδυάζουν δεδομένα από δύο διαφορετικές περιοχές του κώδικα, από το εσωτερικό της συνάρτησης κατά την εκτέλεση της οποίας συμβαίνει το απροσδόκητο γεγονός και από την περιοχή του κώδικα που καλεί αυτήν τη συνάρτηση. Με αυτήν την έννοια, οι εξαιρέσεις δεν υπακούουν στις τρεις αρχές του θεωρήματος του δομημένου προγράμματος. Ωστόσο, είναι αναγκαία η χρησιμοποίησή τους και όλες οι σύγχρονες γλώσσες ενσωματώνουν κάποιο μηχανισμό εξαιρέσεων.

Μια συνέπεια του θεωρήματος του δομημένου προγράμματος είναι η λεγόμενη μοναδική έξοδος (single exit). Σύμφωνα με την αρχή της μοναδικής εξόδου, οι επαναληπτικές δομές και οι συναρτήσεις οφείλουν να τερματίζουν από ένα μόνο συγκεκριμένο σημείο. Σήμερα υπάρχουν διαφορετικές προσεγγίσεις επάνω σε αυτό το θέμα με κυριότερη την πρόωρη έξοδο (early exit). Με βάση την προσέγγιση αυτή, μια συνάρτηση μπορεί να ελέγξει κάποιες συνθήκες και να προχωρήσει σε έξοδο από οποιοδήποτε σημείο. Κάτι τέτοιο είναι ασφαλέστερο να γίνει σε ένα αρχικό στάδιο πριν η συνάρτηση προχωρήσει σε διαχείριση πόρων. Σε πολλές περιπτώσεις, με την πρόωρη έξοδο ο κώδικας γίνεται απλούστερος, ασφαλέστερος και πιο κατανοητός. Η πρόωρη έξοδος εφαρμόζεται σε πολλούς κώδικες όπου θεωρείται χρήσιμη σε αυτό το βιβλίο. Η πρόωρη έξοδος υποστηρίζεται από όλες σχεδόν τις σύγχρονες γλώσσες και για τις επαναληπτικές δομές με τη χρήση των λέξεων-κλειδιών continue και break.

Οι έννοιες αναγνωσιμότητα προγράμματος (code readability), επαναχρησιμοποίηση κώδικα (code reusability) και αφαιρετικότητα (abstraction) σχετίζονται στενά μεταξύ τους αλλά και με το υπόδειγμα του δομημένου προγραμματισμού. Ωστόσο, οι έννοιες αυτές συναντώνται και σε άλλα μοντέλα προγραμματισμού, όπως ο αντικειμενοστρεφής προγραμματισμός. Η συζήτηση εδώ όμως αναφέρεται στις συγκεκριμένες έννοιες και το ειδικό περιεχόμενο που αυτές προσλαμβάνουν στο επίπεδο του δομημένου προγραμματισμού.

Ας δούμε όμως τι σημαίνει αφαιρετικότητα και επαναχρησιμοποίηση κώδικα στην πράξη και στο πλαίσιο της Java. Ας υποθέσουμε πως έχουμε ένα πρόβλημα στα πλαίσια του οποίου χρειάζεται να ψάξουμε σε ένα πίνακα μήκους 10 να διαπιστώσουμε αν υπάρχει ένα στοιχείο ή όχι. Μία λύση είναι να παρεμβάλουμε εκεί όπου χρειάζεται έναν κώδικα που ψάχνει στον συγκεκριμένο πίνακα. Μια καλύτερη λύση όμως είναι να κάνουμε μια συνάρτηση που λαμβάνει παραμετρικά τον πίνακα και το υπό αναζήτηση στοιχείο και επιστρέφει κατά πόσο το στοιχείο περιλαμβάνεται στον πίνακα. Η συνάρτηση αυτή μπορεί να χρησιμοποιηθεί και αλλού όπου είναι χρήσιμη. Η συνάρτηση θα πρέπει να σχεδιαστεί με όσο το δυνατόν μεγαλύτερη αφαιρετικότητα, δηλαδή με όσο το δυνατόν μικρότερη σχέση με το συγκεκριμένο πρόβλημα που αποτέλεσε κίνητρο για την ανάπτυξή της. Με άλλα λόγια, η συνάρτηση δεν θα πρέπει να αξιοποιεί το γεγονός ότι ο πίνακας στον οποίο θέλουμε να ψάξουμε έχει μήκος 10. Αντίθετα, θα πρέπει να γραφεί κατά τρόπο που να μπορούμε να την

καλέσουμε για οποιονδήποτε πίνακα ανεξάρτητα από το μήκος του. Είναι προφανές πως η αφαιρετικότητα συμβάλλει στη δυνατότητα επαναχρησιμοποίησης του κώδικα.

Στο συγκεκριμένο πρόβλημα αναζήτησης μας ενδιαφέρει αν μια τιμή υπάρχει σε έναν πίνακα, επομένως, η συνάρτηση θα μπορούσε να είναι τύπου boolean. Παρόλα αυτά, μπορούμε να αυξήσουμε τη δυνατότητα επαναχρησιμοποίησης της συνάρτησης αν αυτή επιστρέφει ακέραιο ο οποίος στην περίπτωση που βρεθεί το στοιχείο μεταφέρει τη θέση του στον πίνακα, αλλιώς μια τιμή που δεν μπορεί να αποτελεί θέση σε πίνακα, π.χ. μια αρνητική τιμή. Έτσι το πρόγραμμα-πελάτης, δηλαδή η εφαρμογή που θα καλεί τη συνάρτησή μας, μπορεί να την καλέσει απλώς για να ελέγξει αν υπάρχει ή όχι το στοιχείο αλλά μπορεί εφόσον το χρειάζεται να αξιοποιήσει τη θέση του στοιχείου στον πίνακα.

Ας σημειώσουμε εδώ πως η έννοια της δόμησης δεν αφορά στενά την κωδικοποίηση σε μια γλώσσα προγραμματισμού αλλά σχετίζεται και με την ανάλυση (structured analysis) και με τον σχεδιασμό (structured design).

8.2 Τα Στάδια ανάπτυξης

Στην υλοποίηση των εφαρμογών θα διευκολυνθούμε αν ακολουθούμε τα βήματα:

Συλλογή απαιτήσεων (Requirements Gathering). Θα πρέπει να καταγράψουμε με όσο μεγαλύτερη ακρίβεια τις απαιτήσεις των δυνητικών χρηστών της εφαρμογής, δηλαδή τι πρέπει να κάνει η εφαρμογή. Η ανάπτυξη σεναρίων χρήσης (use cases) μπορεί να φανεί πολύ χρήσιμη. Τα σενάρια χρήσης περιγράφουν καταστάσεις κατά τις οποίες το υπό ανάπτυξη λογισμικό μπορεί να είναι χρήσιμο ή υπό μία διαφορετική έννοια, περιγράφουν την ανταπόκριση του λογισμικού σε πιθανές εισόδους του χρήστη. Αποφάσεις σχετικά με το πώς θα επιτευχθούν οι απαιτήσεις ή ποια εργαλεία λογισμικού θα χρησιμοποιηθούν για τον σκοπό αυτό δεν περιλαμβάνονται σε αυτό το στάδιο.

Ανάλυση Απαιτήσεων (Requirements Analysis). Η ανάλυση των απαιτήσεων έχει ως σκοπό να αντιστοιχίσει τις απαιτήσεις των χρηστών σε μια σειρά σαφώς προσδιορισμένων εργασιών του υπό ανάπτυξη συστήματος.

Σχεδιασμός του Συστήματος (System Design). Ο στόχος εδώ είναι ο σχεδιασμός των αυτόνομων ρουτίνων και της σύνθεσής τους με σκοπό την υλοποίηση των εργασιών του συστήματος, όπως αυτές καθορίστηκαν στο στάδιο της ανάλυσης.

Υλοποίηση (Implementation). Εδώ περνάμε στην υλοποίηση των ρουτίνων που εντοπίστηκαν στον σχεδιασμό, δηλαδή στην κωδικοποίησή τους σε ένα πλαίσιο προγραμματισμού.

Έλεγχος (test). Τα λάθη στους κώδικες είναι συχνό και ως έναν βαθμό αναπόφευκτο πρόβλημα. Σε αυτό το στάδιο, θα πρέπει να ελέγξουμε συστηματικά την εκτέλεση και τα αποτελέσματα της εφαρμογής μας με σκοπό να εντοπίσουμε πιθανά προβλήματα.

Επειδή καμία εφαρμογή δεν παράγεται πλήρως ολοκληρωμένη, συχνά τα παραπάνω βήματα επαναλαμβάνονται είτε γιατί αποκαλύπτονται αδύνατα σημεία είτε γιατί προστίθενται επιπλέον απαιτήσεις. Έτσι μιλάμε για κυκλική ανάπτυξη (cyclical development) ή για κύκλο ζωής ανάπτυξης εφαρμογών (Application development life cycle). Συχνά μάλιστα, στο περιβάλλον του επαγγελματικού προγραμματισμού συμπεριλαμβάνονται στα στάδια ανάπτυξης, η διανομή του λογισμικού (deployment) και η παρακολούθησή του στον πελάτη (monitoring).

Στη συνέχεια, βασιζόμενοι σε ένα απλό παράδειγμα προσπαθούμε να δείξουμε πώς μπορούμε να εφαρμόζουμε τα στάδια της ανάπτυξης στο επίπεδο των απλών εφαρμογών αυτού του βιβλίου ώστε από την αρχή να μάθουμε να δομούμε τις εφαρμογές όσο το δυνατόν αποτελεσματικότερα.

8.3 Συλλογή απαιτήσεων

Η συλλογή των απαιτήσεων (requirements gathering) είναι από μόνη της απαιτητική δουλειά. Σε γενικές γραμμές, οι απαιτήσεις χωρίζονται σε δύο κατηγορίες, στις απαιτήσεις του χρήστη (user requirements) και στις λειτουργικές απαιτήσεις (functional requirements).

Στις απαιτήσεις του χρήστη συμπεριλαμβάνονται απαιτήσεις τόσο των άμεσων χρηστών της εφαρμογής όσο και απαιτήσεις άλλων παραγόντων όπως η αγορά, η επιχείρηση για την οποία σχεδιάζεται το λογισμικό, κλπ. Για παράδειγμα, ο χρήστης θέλει να μπορεί να πλοηγείται σε λίστα προϊόντων και να βρίσκει εύκολα το προϊόν που επιθυμεί. Αυτό είναι μια απαίτηση του χρήστη. Όταν ο χρήστης πλοηγείται σε λίστα προϊόντων, θα

πρέπει τα προϊόντα στη λίστα να ταξινομούνται ανάλογα με το τι έχει ήδη αγοράσει ο συγκεκριμένος χρήστης. Αυτό είναι μια λειτουργική απαίτηση. Με άλλα λόγια, οι απαιτήσεις του χρήστη περιγράφουν τι θέλει ο χρήστης από την εφαρμογή, ενώ οι λειτουργικές απαιτήσεις περιγράφουν τι κάνει η εφαρμογή.

Δεν πρόκειται να επεκταθούμε άλλο στο μεγάλο και σημαντικό θέμα της συλλογής απαιτήσεων. Ο αναγνώστης που ενδιαφέρεται για περισσότερες λεπτομέρειες μπορεί να διαβάσει το [4]. Στη συνέχεια, θα υποθέσουμε πως το κείμενο που ακολουθεί περιγράφει τις απαιτήσεις για μια εφαρμογή. Πρόκειται στην ουσία για ένα παράδειγμα αναφορικά με το οποίο θα παρουσιαστούν και οι υπόλοιπες διεργασίες του κύκλου της ανάπτυξης.

Παράδειγμα απαιτήσεων Χρήστη

Τρεις φίλοι, ο John, ο Jim και ο Jack θα παίξουν ένα παιχνίδι ζαριών. Σύμφωνα με τους κανόνες του παιχνιδιού, κατά τη διάρκεια μιας παρτίδας, οι παίκτες ρίχνουν εναλλάξ το ζάρι 10 φορές και μετράνε πόσες φορές έφερε ο καθένας τους άρτια ζαριά. Νικητής είναι εκείνος που έφερε τις περισσότερες άρτιες ζαριές. Αν όμως δύο ή περισσότεροι παίκτες έχουν ισοβαθμήσει, η παρτίδα επαναλαμβάνεται για όλους. Η επανάληψη συνεχίζεται μέχρις ότου να επικρατήσει ένας μοναδικός νικητής.
Οι συγκεκριμένοι παίκτες όμως έχουν τις ιδιομορφίες τους. Ο John είναι ένας τίμιος παίκτης με την έννοια πως κάθε φορά που πετάει το ζάρι έχει την ίδια πιθανότητα να φέρει οποιοδήποτε αποτέλεσμα από 1 έως 6. Από την άλλη μεριά, ο Jim είναι τυχερός, καθώς έχει αποδειχτεί πως όταν πετάει το ζάρι έχει 15% πιθανότητα για κάθε ένα από τα αποτελέσματα 1, 3 και 5, 18% για κάθε ένα από τα αποτελέσματα 2 ή 4 και 19% για να φέρει 6. Επομένως, ο Jim φέρνει άρτια ζαριά με πιθανότητα 55% και περιττή ζαριά με πιθανότητα 45%.
Αντίθετα, ο Jack είναι άτυχος, καθώς φέρνει 2, 4, 6 με πιθανότητα 15% για κάθε ένα από αυτά τα αποτελέσματα, ενώ 1 και 3 φέρνει με πιθανότητα 18% και 5 φέρνει με πιθανότητα 19%. Επομένως, ο Jack φέρνει άρτια ζαριά με πιθανότητα 45% και περιττή ζαριά με πιθανότητα 55%.
Να υλοποιηθεί εφαρμογή που προσομοιώνει ένα παιχνίδι ζαριών μεταξύ των τριών παικτών και αναφέρει τον νικητή.

8.4 Σχεδιασμός

Γενικά υπάρχουν δύο βασικές προσεγγίσεις στον σχεδιασμό ενός προγράμματος. Η από πάνω προς τα κάτω (top-down) και η από κάτω προς τα επάνω (bottom-up).

Σύμφωνα με την top-down, θα πρέπει αρχικά να σχεδιάσουμε την εφαρμογή καταρχάς στο υψηλότερο επίπεδο. Για παράδειγμα, μιλώντας με όρους Java, θα πρέπει πρώτα να σχεδιάσουμε την main. Σε αυτό το επίπεδο πρέπει να γνωρίζουμε τι θα κάνουν οι συναρτήσεις που καλούνται από την main χωρίς όμως να έχουμε καθορίσει τις λεπτομέρειες υλοποίησης τους. Στη συνέχεια, με τον ίδιο τρόπο σχεδιάζουμε κάθε συνάρτηση που καλείται από την main, δηλαδή καταρχάς χρησιμοποιούμε κλήσεις συναρτήσεων που δεν έχουμε ακόμη υλοποιήσει. Έτσι, σύμφωνα με την προσέγγιση top-down, σε κάθε βήμα αποσυνθέτουμε ένα πρόβλημα σε άλλα απλούστερα προβλήματα. Εξ' αυτού του λόγου, η μέθοδος αυτή συχνά ονομάζεται και σταδιακή υλοποίηση (stepwise refinement).

Αντίθετα, η προσέγγιση bottom-up ξεκινά τον σχεδιασμό στο χαμηλότερο επίπεδο και σε κάθε βήμα συνθέτει επιμέρους στοιχεία που οδηγούν σε υψηλότερο επίπεδο, δηλαδή σε σύστημα σταδιακά πιο πολύπλοκο.

Κάθε μια από τις μεθοδολογίες αυτές έχει πλεονεκτήματα και μειονεκτήματα. Η top-down προσέγγιση αργεί σχετικά να παράγει ολοκληρωμένες συναρτήσεις με αποτέλεσμα να καθυστερεί η διαδικασία ελέγχου και άρα η ολοκλήρωση του έργου. Η bottom-up είναι ένα στοίχημα, καθώς είναι δύσκολο να καθοριστούν με ακρίβεια οι λεπτομερείς ανάγκες πριν καθοριστεί το πλαίσιο στο οποίο αυτές θα αξιοποιηθούν. Συχνά, στην πράξη χρησιμοποιείται ένας συνδυασμός των δύο προσεγγίσεων. Για το παράδειγμα του παιχνιδιού ζαριών χρησιμοποιούμε την top-down προσέγγιση.

Αναλύοντας λοιπόν το κείμενο των απαιτήσεων του χρήστη, βλέπουμε πως εκείνο που ζητά είναι η προσομοίωση ενός παιχνιδιού ζαριών και η ανακήρυξη του νικητή. Ένα παιχνίδι όμως ζαριών συνίσταται στην επανάληψη μιας παρτίδας έως ότου ανακηρυχθεί μοναδικός νικητής.

Με βάση αυτήν την πληροφορία, εύκολα μπορούμε να αρχίσουμε τον σχεδιασμό χρησιμοποιώντας διαγράμματα ροής ή ψευδοκώδικα.

```
game:
      do {
```

```
        scores=oneLot;
    } while withdraw(scores);
    return winner(scores);
```

Κώδικας 8.1 *Ψευδοκώδικας για το παιχνίδι ζαριών*

Ο κώδικας 8.1 παρουσιάζει σε μορφή ψευδοκώδικα έναν σχεδιασμό για την υλοποίηση της κύριας απαίτησης του χρήστη. Πράγματι, εδώ ορίζεται μια ρουτίνα που ονομάζεται game και συνίσταται στην επανάληψη μιας παρτίδας ζαριών (oneLot) έως ότου προκύψει μοναδικός νικητής. Σε αυτό το στάδιο δεν μας ενδιαφέρει πώς ακριβώς θα επιτυγχάνει τους στόχους της η oneLot αλλά μόνο πώς αυτή είναι μια διαδικασία που προσομοιώνει μια παρτίδα και επιστρέφει τα σκορ των παικτών. Το ίδιο ισχύει και για την withdraw. Δεν μας αφορούν οι λεπτομέρειες υλοποίησής της αλλά μόνο ότι ανιχνεύει ενδεχόμενη ισοπαλία με βάση μια λίστα από σκορ. Τέλος, η winner λαμβάνει μια λίστα από σκορ τέτοια ώστε σε αυτήν υπάρχει ένα μόνο μέγιστο σκορ και επιστρέφει τον νικητή. Υποθέτουμε πως η winner λαμβάνει λίστα από σκορ στην οποία υπάρχει μία μόνο μέγιστη τιμή, καθώς η λίστα έρχεται από το παιχνίδι το οποίο φτάνει σε αυτό το σημείο μόνο εφόσον δεν ανιχνευθούν ισοπαλίες.

Θα πρέπει τώρα να συνεχίσουμε την ανάλυση και τον σχεδιασμό στο αμέσως επόμενο επίπεδο, δηλαδή να παράγουμε ψευδοκώδικα για τις oneLot, withdraw και winner.

Ας σχεδιάσουμε λοιπόν την παρτίδα, oneLot. Αυτή συνίσταται σε 10 ζαριές που ρίχνουν εναλλάξ, οι τρεις παίκτες. Όμως η ρίψη κάθε παίκτη έχει διαφορετικά χαρακτηριστικά. Επομένως, η ρίψη της ζαριάς θα πρέπει να παραμετροποιηθεί ως προς την κατανομή πιθανοτήτων στη βάση των οποίων υπολογίζεται η ρίψη κάθε παίκτη. Επιπλέον, θα ήταν καλό να παραμετροποιήσουμε την παρτίδα ως προς τον αριθμό ρίψεων. Με αυτόν τον τρόπο θα ανταποκριθούμε πιο εύκολα σε περίπτωση που θα γίνει κάποια σχετική αλλαγή στον αριθμό ρίψεων της παρτίδας. Στον κώδικα 8.2 χρησιμοποιούμε την noOfRolls αντί για τη σταθερά 10.

```
oneLot:
    for (noOfRolls times) {

        rslt = roll(Fair);
        if (rslt is even)
            update score for JOHN;

        rslt = roll(lucky);
        if (rslt is even)
            update score for JIM;

        rslt = roll(unlucky);
        if (rslt is even)
            update score for JACK;

    }
    return scores;
```

Κώδικας 8.2 *Ψευδοκώδικας για μια παρτίδα ζαριών*

Ο ψευδοκώδικας στον κώδικα 8.2 δίνει επαναληπτικά τη δυνατότητα σε κάθε παίκτη να πραγματοποιήσει μία ρίψη και αν το αποτέλεσμα είναι άρτιος ενημερώνει το αντίστοιχο σκορ. Σε αυτό το βήμα προκύπτει η ανάγκη να δούμε σε δεύτερο επίπεδο τον σχεδιασμό της roll και της ενημέρωσης του σκορ κάθε παίκτη.

Συνεχίζοντας όμως σε αυτό το επίπεδο μας μένει να σχεδιάσουμε τον έλεγχο της ισοπαλίας και τον εντοπισμό του νικητή. Πρόκειται για δύο πολύ απλές διαδικασίες. Ο εντοπισμός του νικητή πρέπει απλώς να ψάξει σε μια λίστα με σκορ και να επιστρέψει το μεγαλύτερο, οπότε δεν χρειάζεται να επεκταθούμε περισσότερο σε αυτό. Ο έλεγχος της ισοπαλίας πρέπει να ψάξει σε μια λίστα με σκορ να διαπιστώσει εάν υπάρχουν περισσότερες από μία μέγιστες τιμές. Επομένως

```
withdraw:
    return countOccurances(score, maxScore)>1
```

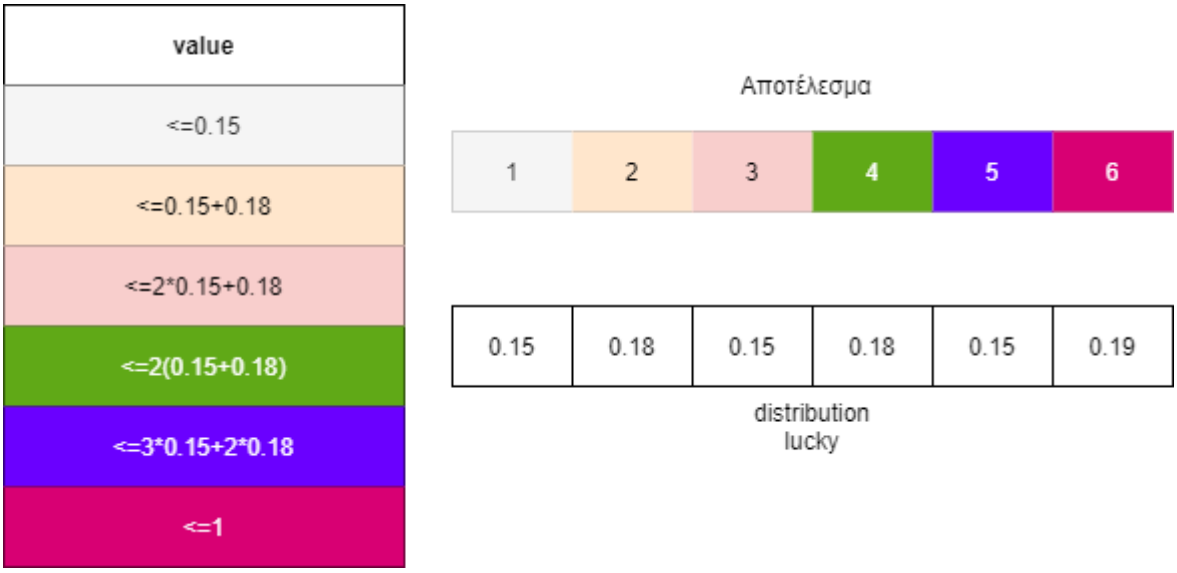
Κώδικας 8.3 *Ανίχνευση ισοπαλίας*

Στο επόμενο επίπεδο θα πρέπει να σχεδιάσουμε καταρχάς τη διαδικασία ρίψης, roll. Η ενημέρωση του σκορ του χρήστη είναι προφανώς πολύ απλή διαδικασία και δεν χρειάζεται περαιτέρω ανάλυση σε αυτό το στάδιο. Καθώς έχουμε τρεις roll οι οποίες επιστρέφουν έναν αριθμό από το 1 έως το 6 με διαφορετική κατανομή πιθανοτήτων για κάθε δυνατό αποτέλεσμα, μπορούμε να σχεδιάσουμε μια διαδικασία παραμετροποιημένη ως προς την κατανομή πιθανοτήτων. Επομένως, θεωρούμε πως η roll λαμβάνει μια λίστα που περιέχει έξι πιθανότητες, μία για κάθε δυνατό αποτέλεσμα μιας ρίψης. Στον ψευδοκώδικα 8.4, η λίστα αυτή ονομάζεται distribution.

```
roll:
     value = Random value from 0 to 1;
     sum = 0;
     rslt=1;
     for (each percentage in distribution) {
          sum += percentage;
          if (value <= sum)
               return rslt;
          rslt++;
     }
}
```

Κώδικας 8.4 *Η διαδικασία προσομοίωσης της ζαριάς*

Το σκεπτικό του κώδικα 8.4 έχει ως εξής: Παράγουμε έναν τυχαίο αριθμό από 0 έως 1, την value. Αν η value είναι μικρότερη ή ίση από την πιθανότητα για αποτέλεσμα 1, επιστρέφουμε 1. Διαφορετικά, αν η value είναι μεγαλύτερη από την πιθανότητα για 1 αλλά μικρότερη από το άθροισμα πιθανοτήτων για 1 και 2, τότε επιστρέφουμε 2. Την ίδια λογική ακολουθούμε για όλα τα δυνατά αποτελέσματα. Με αυτόν τον τρόπο πετυχαίνουμε κάθε αποτέλεσμα με βάση την πιθανότητα που δίνεται στο distribution.



Σχήμα 8.1 *Υπολογισμός της ρίψης με βάση την κατανομή πιθανοτήτων για κάθε δυνατό αποτέλεσμα.*

Στο σχήμα 8.1 οι τιμές της value αντιστοιχίζονται χρωματικά με τα δυνατά αποτελέσματα για την τυχερή κατανομή. Αν η τιμή της value είναι μικρότερη ή ίση του 0.15, η roll επιστρέφει 1. Επομένως, το 1 επιστρέφεται με πιθανότητα 15%. Αν η τιμή της value είναι μικρότερη ή ίση του 0.15+0.18 αλλά δεν είναι μικρότερη ή ίση από 0.15, περίπτωση κατά την οποία η roll θα είχε ήδη επιστρέψει 1, η τιμή επιστροφής είναι 2. Επομένως, η πιθανότητα να επιστρέψει η roll 2 είναι ίση με την πιθανότητα η value να έχει τιμή μικρότερη ή ίση από 0.15+0.18 μείον την πιθανότητα να έχει η value τιμή 0.15. Άρα, η πιθανότητα να επιστρέψει 2 είναι 0.18, δηλαδή σύμφωνη με την κατανομή. Με τον ίδιο τρόπο ερμηνεύεται και ο υπολογισμός των υπόλοιπων τιμών.

8.5 Υλοποίηση

Έχοντας ολοκληρώσει τον σχεδιασμό, μπορούμε να προχωρήσουμε στην υλοποίηση. Στο στάδιο αυτό πρέπει να φροντίσουμε να αξιοποιήσουμε όλες τις δυνατότητες που μας προσφέρει η γλώσσα υλοποίησης, δηλαδή στην προκειμένη περίπτωση, η Java, ώστε ο κώδικάς μας να είναι ευανάγνωστος, απαλλαγμένος κατά το δυνατό από λάθη και εύκολα συντηρήσιμος. Στη συνέχεια παρουσιάζουμε πρώτα την υλοποίηση και μετά την σχολιάζουμε.

```
import java.util.Random;

public class ADiceGame {

    static final double EPSILON = 0.00001;
    private static final Random gen = new Random();
    static final double[] fair = {1/6d, 1/6d, 1/6d, 1/6d, 1/6d, 1/6d};
    static final double[] lucky = {15d/100, 18d/100, 15d/100, 18d/100, 15d/100,
19d/100};
    static final double[] unlucky = {18d/100, 15d/100, 18d/100, 15d/100, 19d/100,
15d/100};
    private static final int JOHN = 0;
    private static final int JIM = 1;
    private static final int JACK = 2;

    public static boolean approximateEquals(double d1, double d2) {
        return Math.abs(d1 - d2) < EPSILON;
    }

    public static double sum(double[] array) {
        double sum = 0;
        for (double d : array) {
            sum += d;
        }
        return sum;
    }

    public static int roll(double[] distribution) {
        if (distribution.length != 6 || !approximateEquals(sum(distribution),
1d)) {
            throw new RuntimeException();
        }
        double value = gen.nextDouble();
        double sum = 0;
        int rslt = 1;
        for (double percentage : distribution) {
            sum += percentage;
            if (value <= sum) {
                return rslt;
            }
            rslt++;
        }
        return 0;
    }

    public static int max(int[] array) {
        int idx = 0;
        int max = array[0];
        for (int i = 1; i < array.length; i++) {
            if (array[i] > max) {
                max = array[i];
                idx = i;
```

```
            }
        }
        return idx;
    }

    public static int cntOccurances(int[] array, int value) {
        int cnt = 0;
        for (int i = 0; i < array.length; i++) {
            if (value == array[i]) {
                cnt++;
            }
        }
        return cnt;
    }

    public static boolean withdraw(int[] score) {
        return cntOccurances(score, score[max(score)]) > 1;
    }

    private static int[] aLot(int noOfRolls) {
        int[] score = new int[3];
        for (int i = 0; i < noOfRolls; i++) {
            int rslt = roll(fair);
            if (rslt % 2 == 0) {
                score[JOHN]++;
            }

            rslt = roll(lucky);
            if (rslt % 2 == 0) {
                score[JIM]++;
            }

            rslt = roll(unlucky);
            if (rslt % 2 == 0) {
                score[JACK]++;
            }
        }
        return score;
    }

    public static int game() {
        final int NOOFROLLS = 10;
        int[] score;

        do {
            score = aLot(NOOFROLLS);
        } while (withdraw(score));
        return max(score);
    }

    private static String winnerName(int winnerId) {

        switch (winnerId) {
            default:
                throw new RuntimeException();
            case JOHN:
                return "John";
            case JIM:
                return "Jim";
            case JACK:
                return "Jack";
```

```
        }
    }

    public static void main(String[] args) {
        int winnerId = game();
        System.out.println("winner is " + winnerName(winnerId));
    }
}
```

Κώδικας 8.5 *Εφαρμογή – Ένα παιχνίδι ζαριών*

Ο κώδικας 8.5 παρουσιάζει την υλοποίηση του παιχνιδιού ζαριών. Στην main καλείται η game που επιστρέφει τον νικητή. Στη συνέχεια, τυπώνεται το όνομά του με τη βοήθεια της winnerName.
Η game υλοποιεί ένα παιχνίδι με συγκεκριμένα χαρακτηριστικά παρτίδας. Έτσι δηλώνει ως τοπική σταθερά τον αριθμό ρίψεων κάθε παρτίδας. Στη συνέχεια, σύμφωνα με τον σχεδιασμό καλεί την aLot μέσα σε επαναληπτική διαδικασία που ελέγχεται από την withdraw. Σε κάθε κλήση, η aLot επιστρέφει έναν πίνακα με τα σκορ των παικτών. Τα σκορ ελέγχει η withdraw. Αν η withdraw δεν ανιχνεύσει ισοπαλία, τερματίζεται ο βρόχος do και επιστρέφεται η θέση του πίνακα των σκορ με τη μεγαλύτερη τιμή. Εκεί βρίσκεται ο νικητής.

Η aLot λαμβάνει ως παράμετρο τον αριθμό των ρίψεων ανά παρτίδα και παίκτη. Δηλώνει τον πίνακα score όπου αποθηκεύονται οι ζαριές με άρτιο αποτέλεσμα για κάθε παίκτη κατά τη διάρκεια της παρτίδας. Προκειμένου να αντιστοιχίζεται κάθε παίκτης ευκολότερα, με περισσότερη ασφάλεια και να είναι το πρόγραμμα ευανάγνωστο, έχουν δηλωθεί οι σταθερές JOHN, JIM και JACK ως 0, 1 και 2. Έτσι γίνεται μια αντιστοιχία μεταξύ των θέσεων στον πίνακα των σκορ με τα ονόματα των παικτών. Μέσα στον βρόγχο for της aLot κάθε παίκτης ρίχνει τη ζαριά του με τη βοήθεια της roll και αν το αποτέλεσμα είναι άρτιο, ενημερώνεται το σκορ του.

Η withdraw, σύμφωνα με τον σχεδιασμό μετράει πόσες φορές στον πίνακα των σκορ υπάρχει το μέγιστο σκορ. Επιστρέφει true μόνο αν βρει περισσότερες από μια μέγιστες τιμές. Την απαιτούμενη μέτρηση υλοποιεί η cntOccurances με τη βοήθεια της max που εντοπίζει τη μέγιστη τιμή.

Η roll λαμβάνει ως παράμετρο έναν πίνακα από πραγματικούς που αναπαριστά την κατανομή των πιθανοτήτων για κάθε δυνατό αποτέλεσμα. Επειδή στο συγκεκριμένο παιχνίδι έχουμε τρεις πιθανές κατανομές, έχουν οριστεί τρεις πίνακες, ο fair, ο lucky και ο unlucky που αναπαριστούν τις τρεις κατανομές. Κάθε πίνακας έχει 6 στοιχεία, καθώς δίνει μια πιθανότητα για κάθε ένα από τα έξι πιθανά αποτελέσματα. Επειδή ρίχνοντας το ζάρι η πιθανότητα να έρθει ένα από τα έξι πιθανά αποτελέσματα είναι 100%, το άθροισμα των στοιχείων κάθε ενός πίνακα πρέπει να είναι 1. Τους ελέγχους αυτούς κάνει η roll. Αν δεν επαληθευτούν, παράγει εξαίρεση. Στη συνέχεια εφαρμόζει τον αλγόριθμο σύμφωνα με τον σχεδιασμό και επιστρέφει το αποτέλεσμα.

Η sum και η approximateEquals είναι προφανείς και έχουν ήδη συζητηθεί.

Προσέξτε πως κάποια μέλη της ADiceGame είναι δημόσια, άλλα ιδιωτικά και άλλα έχουν πρόσβαση πακέτου. Πιο συγκεκριμένα, η EPSILON και οι κατανομές έχουν πρόσβαση πακέτου. Όπως θα δούμε, οι μεταβλητές αυτές θα μας είναι χρήσιμες κατά την ανάπτυξη του ελέγχου του κώδικα και για αυτόν τον λόγο τους δώσαμε πρόσβαση πακέτου, διαφορετικά θα μπορούσαμε να σκεφτούμε να τις ορίσουμε ως ιδιωτικές, καθώς σχετίζονται πολύ στενά με τη συγκεκριμένη εφαρμογή. Η γεννήτρια τυχαίων αριθμών gen είναι ένα αντικείμενο που χρησιμοποιείται στη συγκεκριμένη κλάση για την παραγωγή τυχαίων αριθμών και δεν υπάρχει λόγος να δώσουμε πρόσβαση σε αυτήν έξω από την κλάση. Σε περίπτωση που άλλες κλάσεις επιθυμούν να παράγουν τυχαίους αριθμούς μπορούν να δημιουργήσουν δικές τους γεννήτριες. Τα ονόματα των παικτών αφορούν αποκλειστικά το συγκεκριμένο παιχνίδι και δεν χρειάζεται να έχει πρόσβαση σε αυτά κανείς άλλος.

Επίσης, ως ιδιωτικές έχουν δηλωθεί οι aLOt, game και winnerName. Αυτή η επιλογή οφείλεται στο γεγονός ότι οι τρεις αυτές συναρτήσεις είναι πολύ εξειδικευμένες και αφορούν αποκλειστικά τη συγκεκριμένη εφαρμογή.

Τα υπόλοιπα μέλη της κλάσης έχουν οριστεί ως δημόσια κυρίως με την έννοια πως παρουσιάζουν τέτοια χρησιμότητα ώστε είναι πιθανό να αξιοποιηθούν και από τρίτους.

8.6 Τεκμηρίωση

Η τεκμηρίωση των προγραμμάτων της Java γίνεται με το ειδικό εργαλείο παραγωγής τεκμηρίωσης JavaDoc [3, 4]. Όλες οι προκαθορισμένες κλάσεις της Java είναι τεκμηριωμένες με το JavaDoc. Αν πάτε σε έναν κώδικα στο NetBeans σε ένα σημείο που αναφέρεται το όνομα μιας προκαθορισμένης κλάσης, π.χ. String και πατήσετε